



Numpy

By PrudviKrishna





Numpy - arrays

- NumPy is a fundamental library for scientific computing in Python.
- Support for arrays and matrices, along with a collection of mathematical functions to operate.
- Core operations of np.array (1D, 2D, 3D arrays), arange(), reshape() etc.

np.array()

This function creates an array in NumPy, which can be 1D, 2D, 3D etc. using list, set, tuples

1D Array: A single-dimensional array (vector or 1D tensor).

```
import numpy as np
arr_1d = np.array([1, 2, 3])
print(arr_1d) # Output: [1 2 3]
```

3D Array: A three-dimensional array (3D tensor).

```
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(arr_3d)
# Output:
# [[[1 2]
#      [3 4]]
#
#      [[5 6]
#      [7 8]]]
```

2D Array: A two-dimensional array (matrix or 2D Tensor).

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr_2d)
# Output:
# [[1 2 3]
#      [4 5 6]]
```



Array Attributes

NumPy arrays have several attributes that provide information about the array

shape: Returns the dimensions of the array.

```
arr = np.array([[1, 2], [3, 4]])  
print(arr.shape) # Output: (2, 2)
```

dtype: Returns the data type of the elements.

```
print(arr.dtype) # Output: int64 (or float64 if applicable)
```

ndim: Returns the number of dimensions of the array.

```
print(arr.ndim) # Output: 2
```

#To know shape of the array created

```
import numpy as np
```

```
arr= np.array([[1,2],[3,4]])
```

```
print(arr.size)    # no of elements in the array  
print(arr.shape)   # no of rows and columns in the array  
print(arr.ndim)    # this gives the dimension of the array
```

Create arrays - Numpy Functions

np.arange()

Generates an array of evenly spaced values within a specified range. It is same as range(), creates 1D array.

Syntax: np.arange(start, stop, step)

```
#####
```

```
arr = np.arange(10)
```

```
print(arr) # Output: [0 1 2 3 4 5 6 7 8 9]
```

```
#####
```

```
arr = np.arange(2, 10, 2)
```

```
print(arr) # Output: [2 4 6 8]
```

np.reshape()

Reshapes an array without changing its data. **Convert 1D to 2D or 3D** or etc.

```
#####
```

```
arr = np.arange(6)
```

```
reshaped_arr = arr.reshape(2, 3) or (2,-1)
```

```
#Or Re_arr = np.arange(10).reshape(2,3)
```

```
print(reshaped_arr)
```

```
# Output:
```

```
# [[0 1 2]
```

```
# [3 4 5]]
```

```
#####
```

```
arr = np.arange(8)
```

```
reshaped_arr = arr.reshape(2, 2, 2)
```

```
print(reshaped_arr)
```



Create arrays - Numpy Functions

np.ones() and np.zeros()

Creates an array filled with zeros.

```
arr = np.ones(5)
print(arr) # Output: [1. 1. 1. 1. 1.]
```

```
arr = np.ones((2, 3))
print(arr)
```

```
arr = np.zeros((3, 4))
```

np.linspace()

Generates a specified number of evenly spaced values between two numbers (inclusive).

Syntax: np.linspace(start, stop, num) ; **num is number of values to be generated.**

```
import numpy as np
```

```
# Using np.linspace to generate 5 evenly spaced values b/w 0 & 10
```

```
arr_linspace = np.linspace(0, 10, 5)
print(arr_linspace)
```



Create arrays - Numpy Functions

np.random.rand()

Generates an array of random numbers between 0 and 1.

```
#both rand and random functionality is same random takes tuple as a param
Arr_2d = np.random.rand(2, 3) or array_2d = np.random.random((2, 3))
print(arr)
# Output (example):
# [[0.5488135  0.71518937 0.60276338]
#  [0.54488318 0.4236548  0.64589411]]
```

np.random.randint():

Generates random integers within a specified range.

```
arr = np.random.randint(0, 10, size=(3, 3))
print(arr)
```

```
# Output (example):
# [[5 9 0]
#  [7 2 1]
#  [8 3 4]]
```



Arth Operation - Numpy Functions

np.flatten():

Flattens a multi-dimensional array into a 1D array.

```
arr = np.array([[1, 2], [3, 4]])  
flattened = arr.flatten()  
print(flattened) # Output: [1 2 3 4]
```

np.sum(), np.mean() , np.std() and np.transpose()

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
total = np.sum(arr)  
print(total) # Output: 21
```

```
mean_val = np.mean(arr)  
std_dev = np.std(arr)  
print(mean_val, std_dev)
```

```
transposed = np.transpose(arr)  
print(transposed)
```

Sum along rows (i.e., sum each column), axis=0

```
sum_along_columns = np.sum(arr, axis=0)  
print("Sum along columns (axis=0):", sum_along_columns)
```

Sum along columns (i.e., sum each row), axis=1

```
sum_along_rows = np.sum(arr, axis=1)  
print("Sum along rows (axis=1):", sum_along_rows)
```

Sum along columns (axis=0): [5 7 9]

Sum along rows (axis=1): [6 15]



Accessing arrays- Indexing

Indexing: Select an element from the array at a specific position.

Accessing 1D array

Syntax: arr[index]

```
arr = np.array([10, 20, 30, 40])
element = arr[2] # Output: 30
```

Accessing 2D array

Syntax : arr[rowindex , colindex] or arr[rowindex][colindex]

```
array_2d = np.array([ [1, 2, 3], [4, 5, 6], [7, 8, 9] ])

element = array_2d[1][2] # Output will be 6
or
element = array_2d[1,2] # this is good syntax to use for faster
retrieval
```

Accessing 3D arrays

Syntax: arr[depth][row][column]

```
# Create a 3D array
array_3d = np.array([
    [ # First 2D matrix
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ],
    [ # Second 2D matrix
        [10, 11, 12],
        [13, 14, 15],
        [16, 17, 18]
    ]
])

# Access the element in the second matrix, first row, third column
element = array_3d[1][0][2] # Output will be 12
print(element)
```




Accessing arrays - slicing

Slicing : Array slicing in NumPy allows you to extract specific portions of an array

1D Array Slicing:

```
sliced_array = arr[start:stop:step]
```

```
import numpy as np
```

```
# Create a 1D array
```

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
# Slicing
```

```
print(arr[2:8])    # Output: [2 3 4 5 6 7]
```

```
print(arr[:5])     # Output: [0 1 2 3 4]
```

```
print(arr[5:])     # Output: [5 6 7 8 9]
```

```
print(arr[::-2])   # Output: [0 2 4 6 8]
```

```
print(arr[::-1])   # Output: [9 8 7 6 5 4 3 2 1
```

```
0]
```

2D Array Slicing:

```
sliced_array = arr[start_row:end_row:step, start_column:end_column:step]
```

```
# Create a 2D array
```

```
arr_2d = np.array([[0, 1, 2],
```

```
                  [3, 4, 5],
```

```
                  [6, 7, 8]])
```

```
# Slicing
```

```
print(arr_2d[1, :]) # Output: [3 4 5] (second row)
```

```
print(arr_2d[:, 1]) # Output: [1 4 7] (second column)
```

```
print(arr_2d[0:2, 0:2]) # Output: [[0 1]
```

```
                                     # [3 4]] (sub-array)
```

```
print(arr_2d[::2, ::2]) # Output: [[0 2]
```

```
                                     # [6 8]] (select every second row and column)
```

```
Print(arr_2d[::-1,:])
```



Add/insert Arrays

add a new row

```
import numpy as np

# Original array
arr = np.array([[1, 2], [3, 4]])

# New row to add (reshaped)
new_row = np.array([[5, 6]]) # This is a 2D array with one row

# Add the new row
new_arr = np.append(arr, new_row, axis=0)

print(new_arr)

# New column
new_column = np.array([[5], [6]]) # New column as a 2D array

# Add the new column
new_arr = np.append(arr, new_column, axis=1)

print(new_arr)
```

#insert at a specific position

```
import numpy as np

# Original array
arr = np.array([[1, 2],
                [3, 4],
                [5, 6],
                [7, 8],
                [9, 10]])

# New row to insert
new_row = np.array([99, 100]) # This is a 1D array

# Insert the new row as the second row (index 1)
new_arr = np.insert(arr, 1, new_row, axis=0)

print(new_arr)
```



Update Arrays

```
import numpy as np
```

```
# Create a 2D array (matrix)
matrix = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])
```

```
print("Original matrix:")
print(matrix)
```

Update Row:

```
# Update the second row (index 1)
matrix[1] = [10, 11, 12]
```

```
print("\nMatrix after updating the second row:")
print(matrix)
```

Update Column:

```
# Update the first column (index 0)
matrix[:, 0] = [0, 0, 0] # Set the first column to 0
```

```
print("\nMatrix after updating the first column:")
print(matrix)
```

Update a subarray (2x2 matrix) in the bottom-right corner

```
matrix[1:, 1:] = [[20, 21], # Update elements (1,1), (1,2), (2,1), (2,2)
                  [22, 23]]
```

```
print("\nMatrix after updating the bottom-right subarray:")
print(matrix)
```



Delete Arrays

For 1D Arrays:

```
import numpy as np
```

```
arr_1d = np.array([10, 20, 30, 40, 50])  
modified_1d = np.delete(arr_1d, 2)  
print(modified_1d) # Output: [10 20 40 50]
```

For 2D Arrays:

axis=0: Delete along the rows (removing entire rows).
axis=1: Delete along the columns (removing entire columns).

```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]])
```

```
modified_2d_rows = np.delete(arr_2d, 1, axis=0) # Delete the second row  
modified_2d_cols = np.delete(arr_2d, 0, axis=1) # Delete the first column
```

```
print("Modified 2D array after deleting the second row:")  
print(modified_2d_rows)  
print("\nModified 2D array after deleting the first column:")  
print(modified_2d_cols)
```

Note: The `np.delete()` function returns a new array with the specified elements removed; it does not modify the original array in place



Mathematical Operations

NumPy supports a variety of mathematical operations:

Element-wise Operations:

```
arr = np.array([1, 2, 3])  
arr_plus_one = arr + 1 # Output: [2 3 4]
```

Array Addition

```
arr1 = np.array([[1, 2], [3, 4]])  
arr2 = np.array([[5, 6], [7, 8]])  
result = arr1 + arr2
```

```
# Output: [[ 6  8]  
# [10 12]]
```

Array Subtraction

```
result_sub = arr1 - arr2
```

```
# Output: [[-4 -4]  
# [-4 -4]]
```

Array Multiplication

```
elementwise = arr1 * arr2 print(elementwise)
```

Dot Product: for 2D matrix it is same as matrix multi matmul()

```
arr1 = np.array([[1, 2], [3, 4]])  
arr2 = np.array([[5, 6], [7, 8]])
```

```
dot_product = np.dot(arr1, arr2)
```

Filtering in NumPy

You can filter arrays using Boolean indexing:

Example:

```
arr = np.array([1, 2, 3, 4, 5])  
filtered = arr[arr > 3] # Output: [4 5]
```

```
Multi_filtered = arr[(arr > 3) & (arr < 5)]
```

Boolean Indexing with Single Condition:

- `arr > 3` creates a Boolean array: `[False, False, False, True, True]`.
- `arr[arr > 3]` selects elements where the condition is True, resulting in `[4, 5]`.

Boolean Indexing with Multiple Conditions:

- `arr > 3` produces `[False, False, False, True, True]`.
- `arr < 5` produces `[True, True, True, True, False]`.
- The bitwise AND operator (`&`) combines these conditions: `[False, False, False, True, False]`.
- `arr[(arr > 3) & (arr < 5)]` selects elements where the combined condition is True, resulting in `[4]`.

Logical Operators (and, or) Are Not Allowed:

- In NumPy, you must use `&` for logical AND and `|` for logical OR instead of Python's `and/or`.

A large, abstract graphic consisting of several concentric, overlapping rings in shades of blue, green, and purple, framing the central text.

Thank you